

模块三： 评估指标与策略接口文档

接口说明

该接口用于定义和管理评估方法。评估方法分为系统**内置指标**（如 Ragas 提供的忠实度、答案相关性）和**用户自定义指标**（基于特定 Prompt 或脚本）。这些指标将在创建评测任务时被勾选引用。

1. 评估方法 (Evaluation Method) 实体字段说明

参数名	类型	是否必填	说明
id	Integer	自动生成	指标唯一标识
name	String	是	指标名称，如 Ragas-Faithfulness
description	String	否	指标的详细评价标准说明
category	String	是	方法分类： RULE_BASED (规则), MODEL_BASED (模型判分)
dimension	String	是	评估维度： EFFECTIVENESS (效果), SAFETY (安全), PERFORMANCE (性能)
mode	String	是	评估模式：RESULT (面向结果), PROCESS (面向过程)
output_type	String	是	输出类型：SCORE_0_1 (0-1分), BOOL (布尔值), TEXT (文本评论)
is_builtin	Boolean	是	是否为系统预置指标（内置指标通常不可修改实现逻辑）
parameters	JSON	否	

			参数模板：定义该指标运行时需要配置（如 <code>threshold</code> 阈值）
implementation	JSON	是	执行核心： 内置方法的类名、自定义方法的 Prompt 或脚本代码

基础配置

- 1. 响应格式：所有接口统一返回 `Result<VO>` 对象。
- 2. Host: `localhost:8080`
- 3. 路径前缀: `/api/eval-methods`

接口列表

接口名称	接口描述	Http方法	URL
createMethod	创建自定义评估指标	POST	<code>/api/eval-methods</code>
getMethodList	获取所有可用指标列表	GET	<code>/api/eval-methods</code>
getMethodById	获取具体指标详情	GET	<code>/api/eval-methods/{id}</code>
updateMethod	更新自定义指标（仅限非内置）	PUT	<code>/api/eval-methods/{id}</code>
deleteMethod	删除自定义指标	DELETE	<code>/api/eval-methods/{id}</code>

关键接口请求示例

1. 获取指标列表 (getMethodList)

GET `localhost:8080/api/eval-methods?dimension=EFFECTIVENESS`

说明：前端在创建任务页面，通过此接口拉取所有可选的指标，按维度或分类展示。

Response:

代码块

```
1  {
2      "code": 200,
3      "data": [
4          {
5              "id": 1,
6              "name": "Ragas-Faithfulness",
7              "dimension": "EFFECTIVENESS",
8              "mode": "RESULT",
9              "is_builtin": true,
10             "description": "评估 Agent 的回答是否完全基于检索到的上下文，防止幻觉。"
11         },
12         {
13             "id": 2,
14             "name": "响应耗时",
15             "dimension": "PERFORMANCE",
16             "mode": "PROCESS",
17             "is_builtin": true,
18             "description": "计算从请求发起至收到 done 事件的总时间。"
19         }
20     ],
21     "msg": null
22 }
```

2. 创建自定义指标 (createMethod)

POST `localhost:8080/api/eval-methods`

说明：用户想自己设计一个“评价 Agent 态度是否友好”的指标。通过 `implementation` 字段存入 LLM-as-a-Judge 的 Prompt。

Request Body:

代码块

```
1  {
2      "name": "交互友好度评分",
3      "description": "使用 GPT-4 作为裁判，评价 Agent 在核查过程中的语气是否专业礼
    貌。",
4      "category": "MODEL_BASED",
5      "dimension": "PERFORMANCE",
6      "mode": "PROCESS",
7      "output_type": "SCORE_1_5",
8      "is_builtin": false,
9      "parameters": {
10         "judge_model": "gpt-4o"
11     },
12     "implementation": {
13         "type": "prompt",
14         "content": "你是一名资深语言专家。请评价以下 Agent 的回复语气。1分最差，5分最
    好。回复格式：score: [数字]"
15     }
16 }
```

Response:

代码块

```
1  {
2      "code": 200,
3      "data": { "id": 10 },
4      "msg": "自定义指标创建成功"
5  }
```

接口设计解释

1. 为什么要分 `mode` (RESULT/PROCESS)?

- **RESULT** 指标：后端在收到 Agent 的 `annotation` 或 `done` 事件后才开始计算。
- **PROCESS** 指标：后端需要实时监控 `thinking` 和 `tool_call` 事件。比如“工具调用准确率”这个指标，必须在 Agent 运行过程中通过拦截 `tool_call` 事件来计算。

2. `implementation` 字段

- 如果 `is_builtin` 为 `true`，`implementation` 可能只是一个 ID（如 `ragas_faithfulness_v1`），后端代码里直接调库。
- 如果 `is_builtin` 为 `false`，这里可以存一段 Python 脚本或一段 Prompt。这实现了你需求中的“支持自定义评估指标”。

3. `parameters` 与任务的关联

这个字段定义了“这个指标需要用户输入什么”。例如：

- 一个名为“延迟预警”的指标，它的 `parameters` 可能是 `{"max_latency": "int"}`。
- 用户在创建任务时，系统会提示用户输入这个 `max_latency` 的值（如 5000ms）。

4. 统计字段 `usage_count`

这个字段虽然不直接参与计算，但对于平台管理非常有用。它可以告诉你哪些评测指标是大家最常用的，哪些指标可能已经过时（很久没人用），方便后续清理和优化。